

AI Assisted Coding LAB-02

Roll No : 2303A51541

Batch : 13

Name : A.Sushmitha

Task 1: Statistical Summary for Survey Data

❖ Scenario:

You are a data analyst intern working with survey responses stored as numerical lists.

❖ Task:

Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

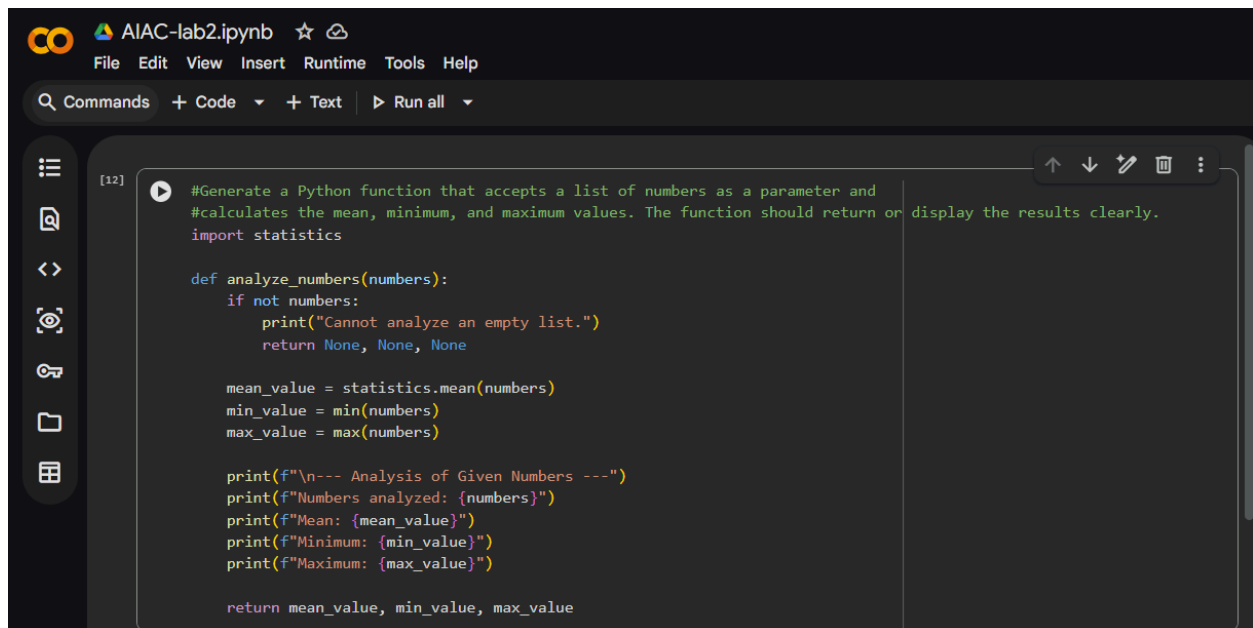
❖ Expected Output:

- Correct Python function
- Output shown in Colab
- Screenshot of Gemini prompt and result

Prompt:

Generate a Python function that accepts a list of numbers as a parameter and calculates the mean, minimum, and maximum values. The function should return or display the results clearly.

Code:

The image shows a Google Colab notebook interface. At the top, the title bar says "AIAC-lab2.ipynb" with a star icon and a share icon. Below the title bar is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Underneath the menu bar is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". The main area of the notebook shows a code cell with a play button icon on the left. The code cell contains a Python function definition and some print statements. The code is as follows:

```
[12] #Generate a Python function that accepts a list of numbers as a parameter and
#calculates the mean, minimum, and maximum values. The function should return or display the results clearly.
import statistics

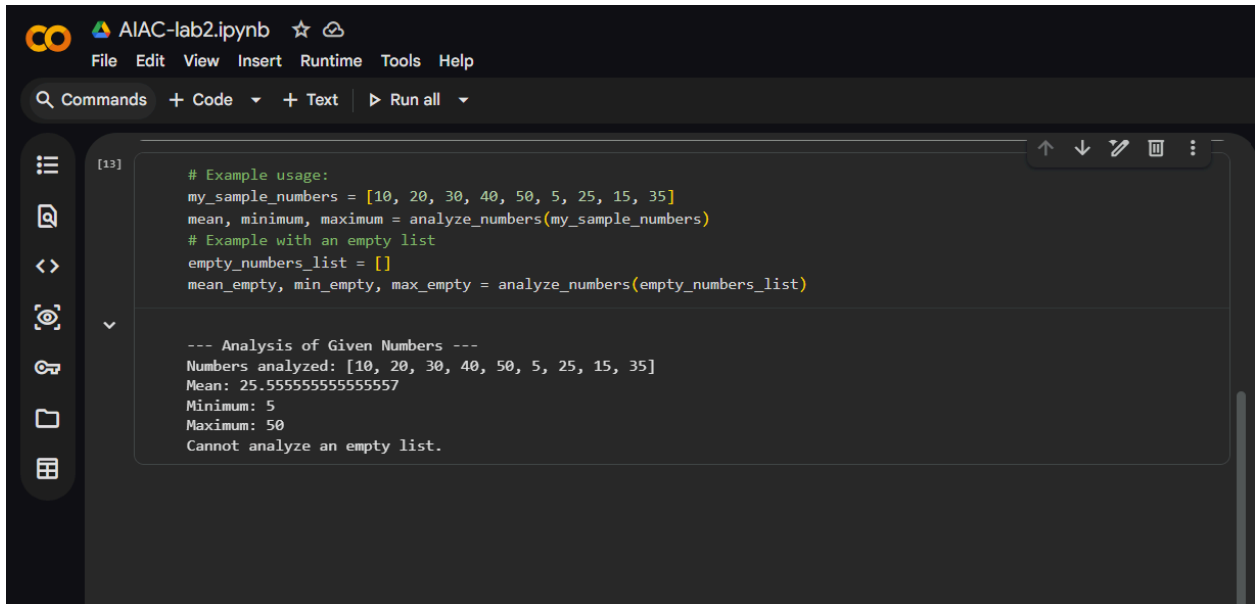
def analyze_numbers(numbers):
    if not numbers:
        print("Cannot analyze an empty list.")
        return None, None, None

    mean_value = statistics.mean(numbers)
    min_value = min(numbers)
    max_value = max(numbers)

    print(f"\n-- Analysis of Given Numbers ---")
    print(f"Numbers analyzed: {numbers}")
    print(f"Mean: {mean_value}")
    print(f"Minimum: {min_value}")
    print(f"Maximum: {max_value}")

    return mean_value, min_value, max_value
```

Output:

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'AIAC-lab2.ipynb' and standard menu options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a search bar and buttons for 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar contains icons for file management and viewing. The main area displays a code cell with the following content:

```
[13]: # Example usage:
my_sample_numbers = [10, 20, 30, 40, 50, 5, 25, 15, 35]
mean, minimum, maximum = analyze_numbers(my_sample_numbers)
# Example with an empty list
empty_numbers_list = []
mean_empty, min_empty, max_empty = analyze_numbers(empty_numbers_list)

--- Analysis of Given Numbers ---
Numbers analyzed: [10, 20, 30, 40, 50, 5, 25, 15, 35]
Mean: 25.555555555555557
Minimum: 5
Maximum: 50
Cannot analyze an empty list.
```

Justification:

The function analyzes a list of survey numbers by calculating the mean, minimum, and maximum values using built-in Python functions. It also checks for empty input to avoid errors and displays the results clearly in Google Colab. This ensures accurate and efficient statistical analysis of the data.

Task 2: Armstrong Number – AI Comparison

❖ Scenario:

You are evaluating AI tools for numeric validation logic.

❖ Task:

Generate an Armstrong number checker using Gemini and GitHub Copilot.

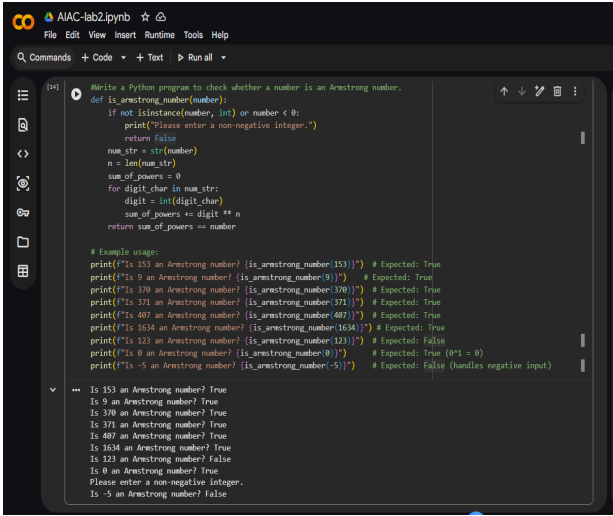
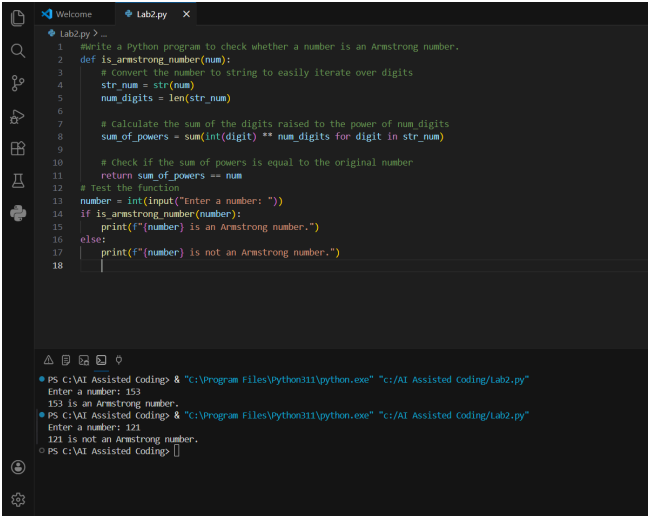
Compare their outputs, logic style, and clarity.

❖ Expected Output:

- Side-by-side comparison table
- Screenshots of prompts and generated code

Prompt:

Write a Python program to check whether a number is an Armstrong number.

Aspect	Google Colab	VS Code
Code		
Output	Tests multiple predefined numbers and prints True / False for each case.	Takes a single user input and prints whether it is Armstrong or not.
Logic Style	Step-by-step approach using loops, explicit variables, and manual summation.	Compact logic using Python's <code>sum()</code> with a generator expression.
Input Handling	No user input – uses hard-coded test values.	Accepts user input from the keyboard.
Error Handling	Handles negative and non-integer input with a message.	No explicit validation for negative or invalid input.
Readability	Very clear for beginners, easy to follow line by line.	Short and clean but slightly advanced for beginners.
Reusability	Function is reusable, but demo code is fixed.	Function is reusable and flexible with user input.

Justification:

The comparison shows how different AI tools generate solutions for the same problem. Google Colab (Gemini) provides step-by-step logic that is easy for beginners to understand and includes basic validation. VS Code (GitHub Copilot) generates concise and efficient code suitable for faster development. This highlights the difference between learning-oriented and productivity-oriented AI tools.

Task 3: Leap Year Validation Using Cursor AI

❖ Scenario:

You are validating a calendar module for a backend system.

❖ Task:

Use Cursor AI to generate a Python program that checks whether a given year is a leap year.

Use at least two different prompts and observe changes in code.

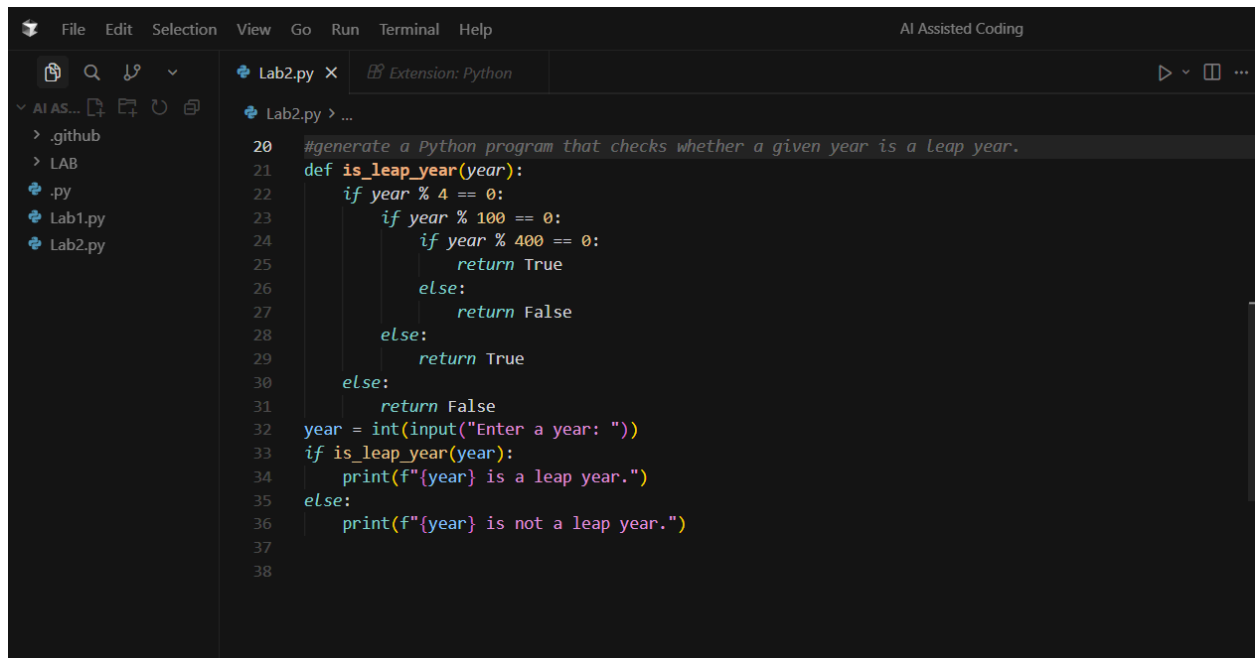
❖ Expected Output:

- Two versions of code
- Sample inputs/outputs
- Brief comparison

Prompt (1):

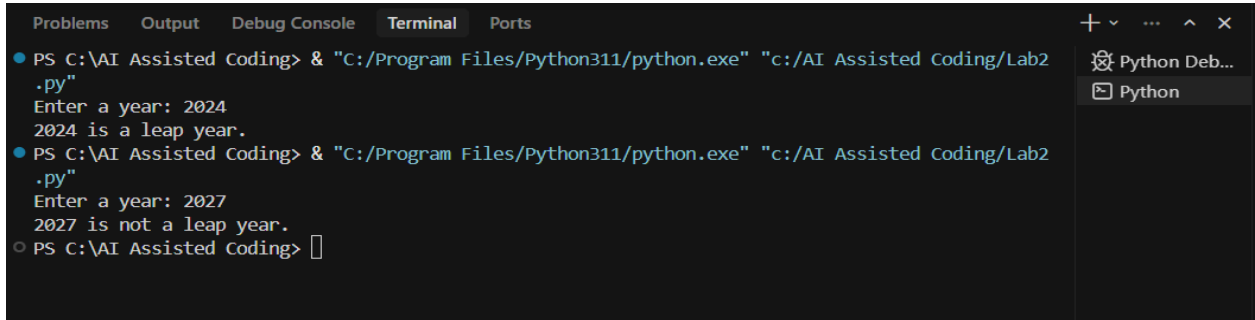
Generate a Python program that checks whether a given year is a leap year.

Code:



```
20 #generate a Python program that checks whether a given year is a Leap year.
21 def is_leap_year(year):
22     if year % 4 == 0:
23         if year % 100 == 0:
24             if year % 400 == 0:
25                 return True
26             else:
27                 return False
28         else:
29             return True
30     else:
31         return False
32 year = int(input("Enter a year: "))
33 if is_leap_year(year):
34     print(f"{year} is a leap year.")
35 else:
36     print(f"{year} is not a leap year.")
37
38
```

Output:

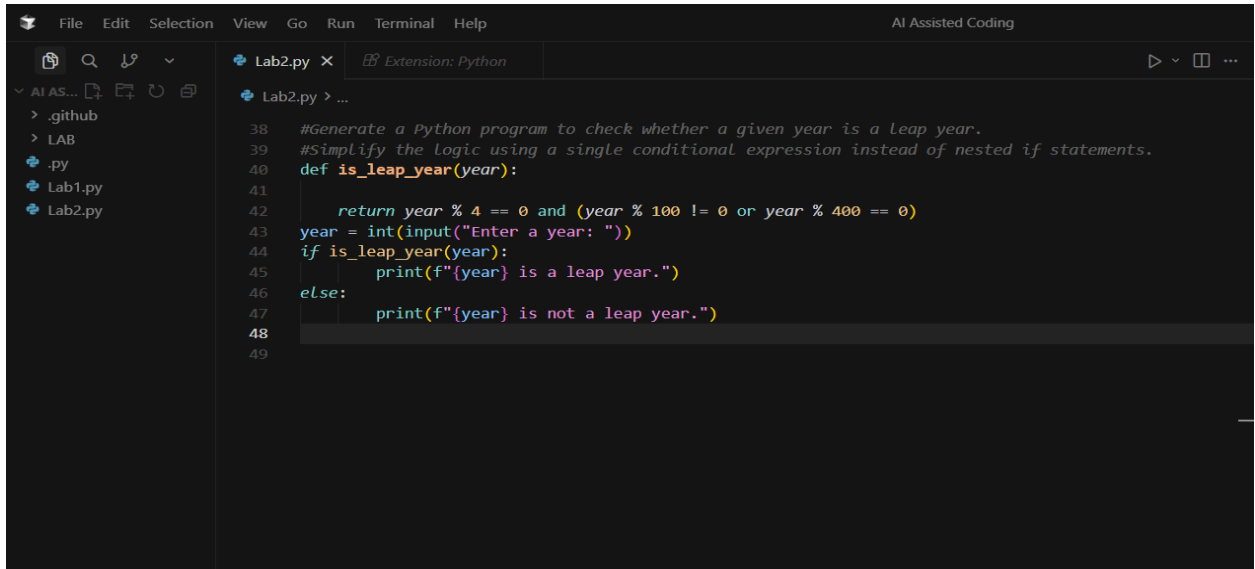


```
PS C:\AI Assisted Coding> & "C:/Program Files/Python311/python.exe" "c:/AI Assisted Coding/Lab2.py"
Enter a year: 2024
2024 is a leap year.
PS C:\AI Assisted Coding> & "C:/Program Files/Python311/python.exe" "c:/AI Assisted Coding/Lab2.py"
Enter a year: 2027
2027 is not a leap year.
PS C:\AI Assisted Coding>
```

Prompt (2):

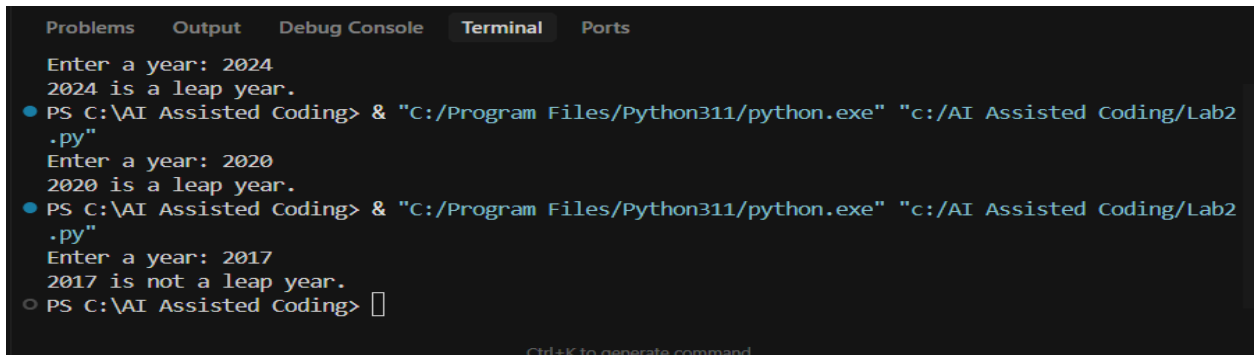
Generate a Python program to check whether a given year is a leap year. Simplify the logic using a single conditional expression instead of nested if statements.

Code:



```
#Generate a Python program to check whether a given year is a Leap year.
#Simplify the Logic using a single conditional expression instead of nested if statements.
def is_leap_year(year):
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
year = int(input("Enter a year: "))
if is_leap_year(year):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
```

Output:



```
Enter a year: 2024
2024 is a leap year.
PS C:\AI Assisted Coding> & "C:/Program Files/Python311/python.exe" "c:/AI Assisted Coding/Lab2.py"
Enter a year: 2020
2020 is a leap year.
PS C:\AI Assisted Coding> & "C:/Program Files/Python311/python.exe" "c:/AI Assisted Coding/Lab2.py"
Enter a year: 2017
2017 is not a leap year.
PS C:\AI Assisted Coding>
```

Comparison of Two Leap Year Programs:

Feature	Prompt(1) – Nested If Method	Prompt(2) – Single Condition Method
Logic Style	Uses multiple nested <code>if-else</code> blocks	Uses one boolean expression
Lines of Code	More lines	Fewer lines
Readability	Harder to read due to nesting	Clean and easy to understand
Performance	Slightly slower due to multiple checks	Faster due to single evaluation
Maintainability	More complex to modify	Easy to modify
Pythonic Style	Traditional logic	Pythonic and optimized

Justification:

Two different prompts were given to Cursor AI to generate leap year validation code, resulting in different coding styles. The first version uses nested if-else statements, making the logic clear and easy to understand step by step. The second version simplifies the same logic into a single conditional expression, reducing code length and improving readability. Both versions produce the same correct output for all inputs. This demonstrates how prompt variation can influence code structure and optimization.

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ Scenario:

Company policy requires developers to write logic before using AI.

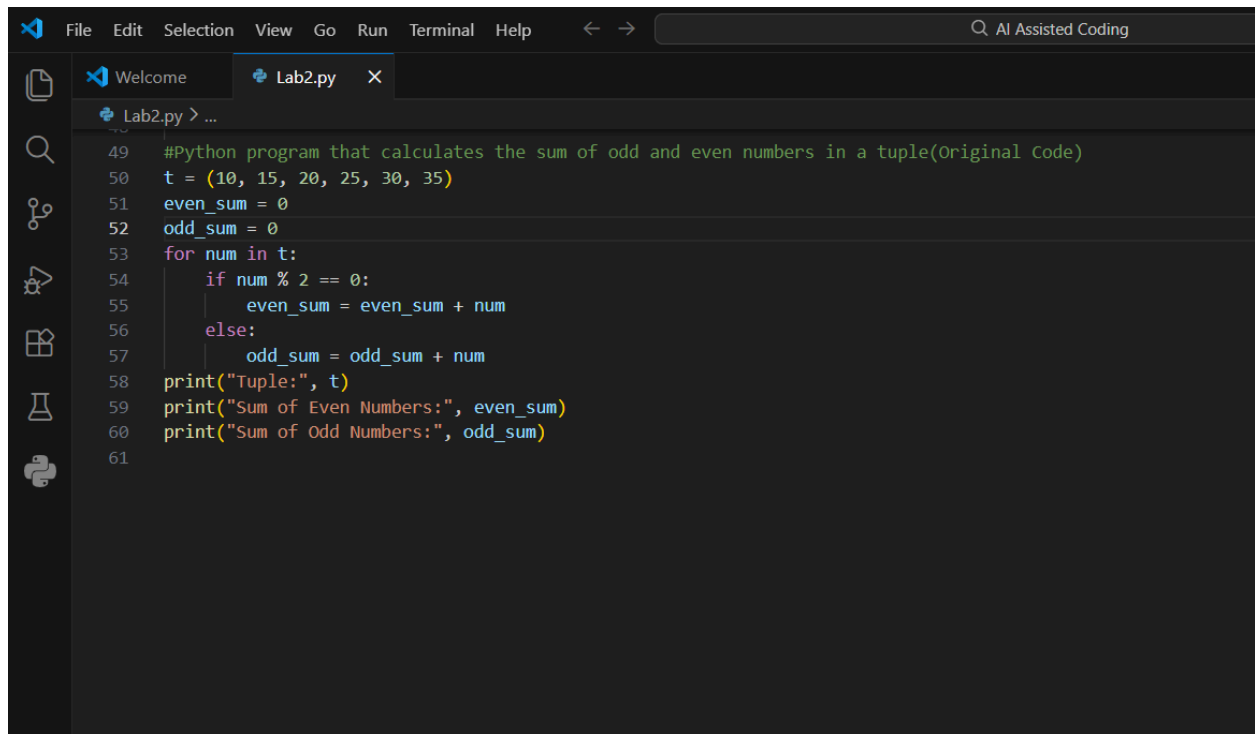
❖ Task:

Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

❖ Expected Output:

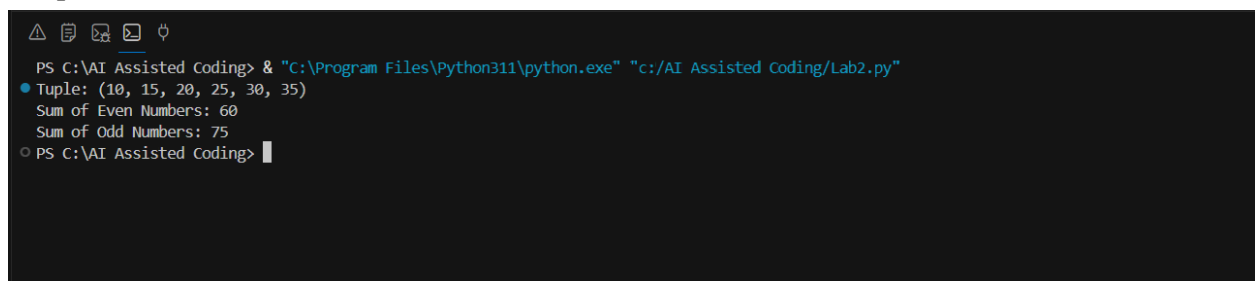
- Original code
- Refactored code
- Explanation of improvements

#Python program that calculates the sum of odd and even numbers in a tuple(My Code)



```
File Edit Selection View Go Run Terminal Help  AI Assisted Coding
Welcome Lab2.py X
Lab2.py > ...
49 #Python program that calculates the sum of odd and even numbers in a tuple(Original Code)
50 t = (10, 15, 20, 25, 30, 35)
51 even_sum = 0
52 odd_sum = 0
53 for num in t:
54     if num % 2 == 0:
55         even_sum = even_sum + num
56     else:
57         odd_sum = odd_sum + num
58 print("Tuple:", t)
59 print("Sum of Even Numbers:", even_sum)
60 print("Sum of Odd Numbers:", odd_sum)
61
```

Output:

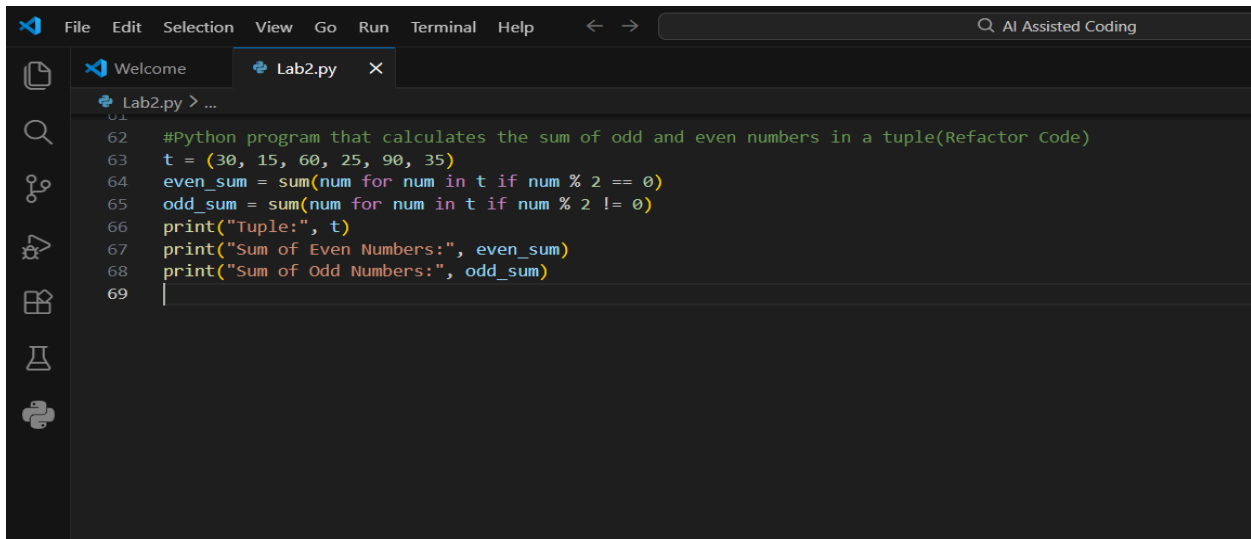


```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab2.py"
• Tuple: (10, 15, 20, 25, 30, 35)
  Sum of Even Numbers: 60
  Sum of Odd Numbers: 75
○ PS C:\AI Assisted Coding>
```

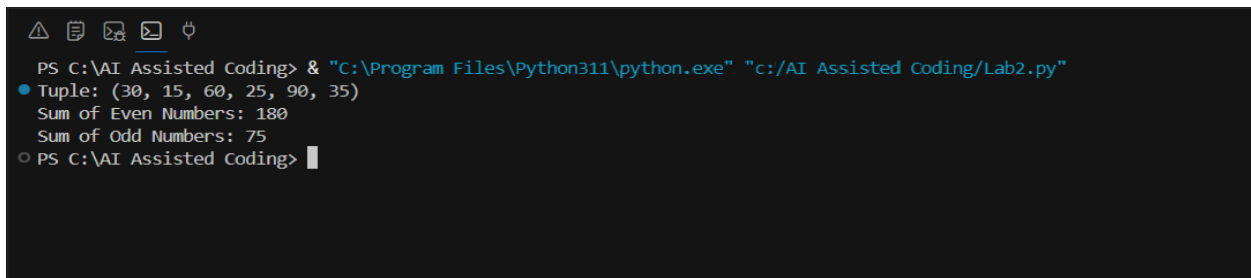
Prompt:

Python program that calculates the sum of odd and even numbers in a tuple(Refactor Code).

Code:



Output:



Improvement Comparison Table (Using GitHub Copilot):

Aspect	Original Code	Refactored Code
Logic Style	Uses loop and manual addition with variables.	Uses <code>sum()</code> with generator expressions.
Code Length	More lines and repetitive statements	Shorter and more compact code.
Readability & Efficiency	Clear but slightly lengthy and slower due to manual processing.	Cleaner, easier to read, and more efficient using built-in functions.

Justification:

Using GitHub Copilot, the original loop-based program was refactored by applying Python's built-in `sum()` function with generator expressions. This reduced the number of lines and eliminated unnecessary variables, making the code cleaner and easier to maintain. The logic became more readable and efficient while producing the same output. This demonstrates how AI tools can improve code quality without changing functionality.